



# Environmental factors analysis and comparison affecting software reliability in development of multi-release software



Mengmeng Zhu, Hoang Pham\*

Department of Industrial and Systems Engineering, Rutgers University, Piscataway, NJ 08854 United States

## ARTICLE INFO

### Article history:

Received 15 December 2016

Revised 9 May 2017

Accepted 25 May 2017

Available online 20 June 2017

### Keyword:

Environmental factors

Multi-release software

Principle component analysis

Multiple linear regression

Lasso regression

Stepwise backward elimination

## ABSTRACT

As the application of the principles of agile and lean software development, software multiple release becomes very common in the modern society. Short iteration and short release cycle have driven the significant changes of the development process of multi-release software product, compared with single release software product. Thus, it is time to conduct a new study investigating the impact level of environmental factors on affecting software reliability in the development of multi-release software to provide a sound and concise guidance to software practitioners and researchers. Statistical learning methods, like principle component analysis, stepwise backward elimination, lasso regression, multiple linear regression, and Tukey method, are applied in this study. Comparisons regarding significant environmental factors during the whole development process, principle components, significant environmental factors in each development phase and significance level of each development phase between the development of single release software and multi-release software are also discussed.

© 2017 Elsevier Inc. All rights reserved.

## 1. Introduction

As software systems are more deeply embedded in our everyday life, the dependence of our modern society on complex, intelligent and reliable large-scale software system is rapidly growing (Mohanty et al., 2013). Meanwhile, the possibility of carrying more faults in the large-scale software system is higher than decades ago. Software failures are becoming increasingly common in the field environment as the increasing complexity of software product (Sahoo et al., 2010). In order to continually align with the fast-changing customer's requirement and provide more reliable product to the market, most companies will release multiple versions of the software product since it is unlikely to deliver all the features in a single release and satisfy all the constraints within limited resource (Kapur et al., 2011; Saliu and Ruhe, 2005). The principles of multi-release software are: adding new features in the next release and fixing the remaining faults from previous release due to the fact that bug-free software product is not likely to be delivered in any release (Maurice et al., 2006). Software multiple releases not only makes company easily balance the competing stakeholder's demand and benefits but also increases reliability and customer satisfaction level during each release (Svahnberg et al., 2010; Ruhe and Momoh, 2005). It is very pragmatic yet interesting to inves-

tigate what are the significant environmental factors on affecting software reliability during the development of multi-release software.

Although no general definition has been given to define what are the environmental factors affecting software reliability during software development process, there have been a number of related works in the literature defined different types of factors in software development from various perspectives. For instance, Misra et al. (2009) identified some success factors in agile software development based on data analysis technique. Chow and Cao (2008) provided a survey study on the critical success factor of agile software development using quantitative approach. Clarke and O'Connor (2012) studied the situational factors affecting software development process. Zhang and Pham (2000) defined 32 environmental factors to capture the characteristics affecting software reliability during software development process for single release software. Another exploratory analysis is also provided (Zhang et al., 2001) to further analyze the detailed relation of these environmental factors. Furthermore, Zhu et al. (2015) presented a comparison analysis to compare the changes of these 32 environmental factors affecting software reliability after Zhang and Pham (2000) first proposed this idea. However, these researches mainly focus on the development process for single release software. To the extent of our knowledge, we have not seen any research studied the impact of the environmental factors affecting software reliability during the development of multi-release software.

\* Corresponding author.

E-mail addresses: [hopham@soe.rutgers.edu](mailto:hopham@soe.rutgers.edu), [hopham@rci.rutgers.edu](mailto:hopham@rci.rutgers.edu), [hoang84pham@gmail.com](mailto:hoang84pham@gmail.com) (H. Pham).

### 1.1. Research motivation

In today's fast-changing technology environment, it is unlikely to include all the required features in a single release software. Most software companies will modify the parts of the existing modules to extend the current functionality and usability by adding new features in the next release and correcting the issues from previous release (Gorschek and Davis, 2008). The importance of software multiple release has been brought to our attention (Febrero et al., 2016). Compared with traditional release cycle, the modern software release cycle is shortened from traditional 18-month to 3-month (Al-Emran and Pfahl, 2007). Moreover, the challenges caused by modern realities are more wide-spreading. The rapid-changing market, the increasing involvement of end-users during software development and the new competitive threats all require reliable products (Beck et al., 2001). Hence, how to assure reliability and quality for multi-release software in such a challenging environment is critical to the company and research institution.

The resources and constraints for the development of multi-release software are different. Software development team needs to select the corresponding features presented in the next release with respect to customers' feedback and market requirements. Since software will be released in increments for multi-release software, thus, coupling this concept with other principles such as continuous unit testing and pair programming will better arrange the cost distribution (Beck et al., 2001).

During the past decades, software version planning and release has been studied by many researchers. Szoke (2011) developed a staged-delivery global optimized model for agile release planning. Li et al. (2014) proposed a multi-objective optimization technique to optimize three main objectives with respect to cost, revenue, and uncertainty for robust next release problem. Etgar et al. (2017) explored several optimization approaches to determine the content and release date for each release in order to provide optimal net present value (NPV). In addition, software reliability model related to software multiple release is also investigated by some researchers. Hu et al. (2011) proposed a software reliability model considering the effect of multiple releases on the fault detection process in software development. Yang et al. (2016) incorporated fault detection and fault correction process in multi-release software reliability modeling. Time delay is assumed in fault repair after detecting faults. However, all of the past research related to release planning or multi-release software reliability modeling did not investigate the impact of environmental factors on software reliability in the development of multi-release software.

Therefore, it is plausible to conduct a new study to investigate the impact of these environmental factors on software reliability for multi-release software and compare the difference between single release software and multi-release software.

### 1.2. Research objectives

As discussed in the previous section, the development process for multi-release software is different with single release software. For example, how to select the features in which release and how to determine the removal percentage of detected software faults. This study aims to revisit the environmental factors in terms of their impact on software reliability for multi-release software. This study is carried out by conducting a survey of environmental factors affecting software reliability for the next release's development.

First, we need to find out what are the significant environmental factors affecting reliability in the development of multi-release software. Secondly, what are the correlations between environmental factors and is it possible to reduce the dimension of

those variables to provide concise and sound information for researchers and practitioners? Moreover, the significant environmental factors in each development phase and the significance level for each development phase are also investigated in order to provide a helpful time/resource allocation matrix for software development team. Thirdly, we also compare the significant environmental factors during the whole development process, principle components, the significant environmental factors in each development phase, significance level of each development phase between the development of single release software and multi-release software. At the end of this study, other statistical method in terms of variable selection is also applied in this paper to provide an insightful matrix for readers according to their selection priority. Software practitioners can choose the results coming from different methodologies based on their requirements.

Section 2 briefly describes how we collected the data for this survey. Section 3 presents the detailed analysis for the environmental factors and each development phase in the development of multi-release software. Section 4 compares the new findings for multi-release software in this study with the previous findings for single release software (Zhu et al., 2015). Section 5 reveals the significant environmental factors based on other variable selection method. Section 6 concludes this study and discusses the sound and pressing suggestion for the future research.

## 2. Data collection

In order to align with the latest survey data analysis focused on the significant environmental factors on affecting software reliability during the development of single release software (Zhu et al., 2015) and maintain the similarity for the comparison of environmental factors, we still use the same 32 environmental factors defined in Zhang and Pham (2000).

45 survey responses are collected from various industry including computer software, internet, banking, semiconductor, online retailing, financial service, IT service & consulting and research institution. Participants ranked the environmental factors based on the impact of each environmental factor on software reliability assessment during the development of multi-release software. The software development experiences, software applications, years of experience, and job title are also provided from the participants. All the participants are currently working in IT Department in different industry or working on software development in high-tech company in favor of the validity and reliability of the survey response. Hence, we are expecting the survey response data is valid and reliable.

## 3. Findings and results

### 3.1. Environmental factors analysis

#### 3.1.1. Relative weighted method

This survey investigation includes 32 environmental factors. In terms of their impact level on software reliability assessment during the development of multi-release software, participants need to provide a rank ranging from 1 to 7 for each environmental factor. For example, if participants think one environmental factor is extremely important on affecting software reliability for the next release's development, they can rank 7; on the contrary, if participants think one environmental factor does not have a significant impact on affecting software reliability during the development of next release, they may rank 1.

Relative weighted method is applied in this study to provide intuitive information regarding the impact of each environmental factor affecting software reliability. The summation of the ranking for all 32 environmental factors is from 0 to 224. In this study,

**Table 1**  
Environmental factors ranking by relative weighted method.

| Rank | Factor | Description                                    | Normalized weight |
|------|--------|------------------------------------------------|-------------------|
| 1    | f1     | Program complexity                             | 0.039087909       |
| 2    | f11    | Requirement analysis                           | 0.038455884       |
| 3    | f8     | Frequency of program specification change      | 0.036885309       |
| 4    | f22    | Testing effort                                 | 0.0361219         |
| 5    | f12    | Relationship of detailed design to requirement | 0.03557156        |
| 6    | f4     | Amount of programming effort                   | 0.035339928       |
| 7    | f25    | Testing coverage                               | 0.034279917       |
| 8    | f18    | Program workload (stress)                      | 0.034200235       |
| 9    | f6     | Percentage of reused modules                   | 0.03417967        |
| 10   | f5     | Level of programming technologies              | 0.034081437       |
| 11   | f15    | Programmer skills                              | 0.033737498       |
| 12   | f21    | Testing environment                            | 0.033712626       |
| 13   | f23    | Testing resource allocation                    | 0.033061158       |
| 14   | f10    | Design methodology                             | 0.032923068       |
| 15   | f24    | Testing methodologies                          | 0.032877178       |
| 16   | f19    | Domain knowledge                               | 0.032722566       |
| 17   | f27    | Documentation                                  | 0.032237781       |
| 18   | f3     | Difficulty of programming                      | 0.032026308       |
| 19   | f26    | Testing tools                                  | 0.031219573       |
| 20   | f14    | Development management                         | 0.031112885       |
| 21   | f16    | Programmer organization                        | 0.03065609        |
| 22   | f2     | Program categories                             | 0.030317901       |
| 23   | f20    | Human nature                                   | 0.030067289       |
| 24   | f17    | Development team size                          | 0.029361362       |
| 25   | f9     | Volume of program design documents             | 0.029125828       |
| 26   | f13    | Work standards                                 | 0.028696588       |
| 27   | f28    | Processors                                     | 0.02485165        |
| 28   | f32    | System software                                | 0.024722279       |
| 29   | f29    | Storage devices                                | 0.024614444       |
| 30   | f7     | Programming language                           | 0.02261143        |
| 31   | f30    | Input/output devices                           | 0.021177569       |
| 32   | f31    | Telecommunication devices                      | 0.019963182       |

**Table 2**  
Eigenvalue and proportion of the principle components.

| Component | Eigenvalue | Proportion of variance | Cumulative proportion |
|-----------|------------|------------------------|-----------------------|
| PC1       | 3.9836     | 0.3821                 | 0.3821                |
| PC2       | 1.2567     | 0.1303                 | 0.5124                |
| PC3       | 1.1596     | 0.1273                 | 0.6397                |
| PC4       | 1.0539     | 0.1113                 | 0.751                 |
| PC5       | 0.8239     | 0.0832                 | 0.8342                |
| PC6       | 0.6142     | 0.0654                 | 0.8996                |
| PC7       | 0.4195     | 0.0429                 | 0.9425                |
| PC8       | 0.2871     | 0.0241                 | 0.9666                |
| PC9       | 0.2418     | 0.0192                 | 0.9858                |
| PC10      | 0.1596     | 0.0141                 | 1.0000                |

analyze environmental factors. Since so many environmental factors are investigated in this study and it is unlikely to apply all of them in practice or software reliability modeling, thus how to reduce the dimension of environmental factors is brought to our attention especially some of the environmental factors are correlated.

Principle component analysis (PCA) is applied to reduce the dimension of the environmental factors. PCA is a statistical method commonly used to reduce the dimensionality of a dataset while retaining as much of the variation present in the dataset as possible (James et al., 2013). This technique often brings out strong pattern in the dataset and makes dataset easy to explore and visualize. The top 10 most important environmental factors based on the relative weighted method are selected to perform principle component analysis.

The top 10 environmental factors are f1, f11, f8, f22, f12, f4, f25, f18, f6, f5. We can notice that the ranking is quite different compared with the findings from Zhu et al., (2015). The group of significant environmental factors affecting software reliability is also different between the development of multi-release software and single release software. R software is utilized to perform principle analysis in this study. The covariance matrix of  $10 \times 10$  will be calculated in order to obtain the eigenvalues and eigenvectors. Eigenvalues, from the highest to the lowest, are listed in Table 2 in terms of their impact level on the principle components.

As seen from Table 2, four principle components will be retained, which are PC1, PC2, PC3, PC4. The first four principle components can address over 75% of the data variation. 38.2% of the data variation can be explained by the first principle component; about 13% of the data variation can be explained by the second principle component. Likewise, the third and the fourth principle component are able to represent 12.7% and 11.1% of the data variation, respectively. All other principle components only can represent less than 10% of the variation, respectively. For PC5, PC6, PC7, PC8, PC9, PC10, the compensation of adding one more principle component is considerably high according to the contribution it can bring in.

After retaining the principles components for environmental factors during the development of multi-release software product, the next question we want to discuss is the contribution of each environmental factor to each principle component. Specifically, each principle component is the linear combination of the environmental factors. Table 3 shows the environmental factors which are strongly-correlated with the principle components. The loading coefficient is utilized to measure the contribution of each environmental factor to the principle components. Relationship of detailed design to requirement is highly correlated with the first principle component; amount of programming effort is highly correlated with the second principle component; program complexity and frequency of specification change are highly correlated with the third and fourth principle component, respectively.

the summation for all survey responses ranges from 87 to 191. The equation for normalization each score in their original survey is given as

$$W_{ij} = \frac{r_{ij}}{\sum_{i=1}^N r_{ij}}$$

where  $r_{ij}$  represents the score of each environmental factor in its original survey, which can be explained as the score of the  $i^{\text{th}}$  factor in the  $j^{\text{th}}$  survey.  $N$  is the total number of factors filled up by the  $j^{\text{th}}$  participant in the  $j^{\text{th}}$  survey. Since all the participants filled up the survey form in this investigation, thus  $N$  is 32 for all survey responses. We can easily find the final weight for the  $i^{\text{th}}$  factor by

$$W_i = \frac{\sum_{j=1}^L W_{ij}}{L}$$

where  $L$  is the number of complete form, which is 45 in this study.

As a result, we are able to calculate the normalized weight for each environmental factor and find their ranking, as shown in Table 1. Program complexity becomes top 1 on the ranking, which has higher impact on affecting software reliability for the development of next release than other environmental factors. In general, the higher rank of an environmental factor in Table 1, the higher impact on software reliability in the development of multi-release software.

### 3.1.2. Principle component analysis for environmental factors

In order to keep the similarity of this study with the previous study (Zhu et al., 2015) and compare what is the difference between these two studies with respect to single release software and multi release software, we are using the same methodology to

**Table 3**  
Principle component associated with strong-correlated environmental factors.

| Principle component | EFs | Description                                    | Loading coefficient |
|---------------------|-----|------------------------------------------------|---------------------|
| PC1                 | f12 | Relationship of detailed design to requirement | −0.399              |
|                     | f25 | Testing coverage                               | −0.383              |
|                     | f22 | Testing effort                                 | −0.383              |
|                     | f11 | Requirement analysis                           | −0.323              |
|                     | f5  | Level of programming technologies              | −0.294              |
| PC2                 | f18 | Program workload (stress)                      | 0.52                |
|                     | f4  | Amount of programming effort                   | −0.308              |
| PC3                 | f6  | Percentage of reused modules                   | −0.526              |
|                     | f1  | Program complexity                             | 0.509               |
| PC4                 | f8  | Frequency of program specification change      | 0.629               |

### 3.1.3. Hypothesis testing

There is a common question that software practitioner often brings up: do these 32 environmental factors have the same impact on software reliability assessment during the development of multi-release software? In this section, one-way ANOVA (Analysis of Variance) is applied to compare the significance level of those environmental factors. From the R output, F-statistic for the environmental factors analysis is 8.342 with a  $p$ -value less than  $2e^{-16}$ ; while the value of  $F_{31, 1408}$  is 1.459. Statistically, the null hypothesis is rejected when  $F$  value is larger than  $F$  critical value of 1.459. Therefore, the significance level of each environmental factor on software reliability assessment is different during the development of multi-release software.

### 3.1.4. Correlation analysis

If researchers desire to find out the linear relationship between any pair of these 32 environmental factors, principle component analysis cannot satisfy this purpose. In order to provide a clear and sound framework of the linear relationship for all 32 environmental factors, correlation analysis, a measure of linear association between two environmental factors, is also applied in this study to find out the strength and direction of each relation. Table 4 (see Appendix) presents the Pearson's  $r$  associated with each pair of environmental factors. The range for Pearson's  $r$  is from  $-1$  to  $1$ . The higher of the absolute value of a Pearson's  $r$ , the stronger linear correlation of these two environmental factors. Due to the limitation of the page, we only choose the absolute value of Pearson's  $r$  is larger than  $0.45$  to present in the table.

## 3.2. Development phase analysis

### 3.2.1. Significance level of each development phase

As described in Zhang and Pham (2000), 32 environmental factors can be categorized into five development phases during software development, general, analysis and design, coding, testing, and hardware system. We are interested to know whether these five development phases have the same impact on software reliability assessment during the development of multi-release software. Thereafter the comparison between the development of single release software and multi-release software in terms of the significance level of each development phase can be made in Section 4. One-way ANOVA method is utilized to compare the significance level. The hypothesis is present as follows.

$$H_0 : \mu_{\text{General}} = \mu_{\text{Analysis\&design}} = \mu_{\text{Coding}} = \mu_{\text{Testing}} = \mu_{\text{Hardware system}}$$

$$H_a : \text{not all equal}$$

From the R output, the F-statistic for the development phase is 35.98 with  $p$ -value less than  $2e^{-16}$ . The F critical value,  $F_{4, 1435}$ , is 2.378 from F table, which is much less than 35.98. Therefore, during the development of multi-release software, the impact of each development phase on software reliability is different.

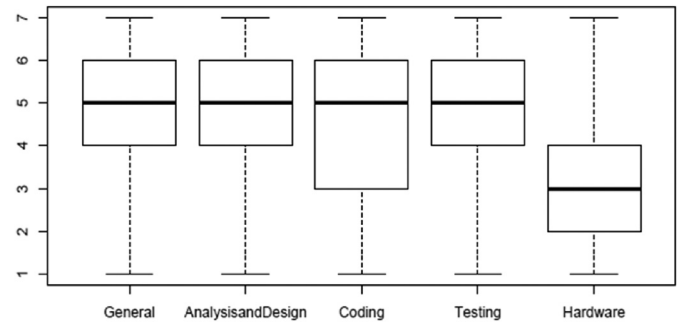


Fig. 1. Boxplot for each development phase.

Fig. 1 shows the box plot for each development phase. General phase, analysis and design phase, coding phase, and testing phase have the similar mean value. In addition, Tukey method is utilized to group different development phase based on the mean value of each phase. From the R output of Tukey multiple comparison of mean, we can consider general, analysis and design, coding and testing as a group. The grouping distribution is quite different with the previous findings from Zhu et al. (2015), for which, we will discuss in Section 4.

### 3.2.2. Significant environmental factors in each development phase

Backward elimination provides an efficient alternative to select the best subset, thereby backward elimination method is applied to select the significant environmental factors within each development phase on affecting software reliability during the development of multi-release software (James et al., 2013). Why do we need to address the significant environmental factors for each development phase? Software development is a risky and expensive process (Adolph et al., 2012). Since there is limited time and resource for developing each software release, how to optimally improve the reliability within limited resource is critical for software development team. Thus, the significant factors within each phase can provide guidance for the software developers to optimally allocate the resource regarding reliability improvement.

For the variable selection process, we use reliability value obtained from the survey as a response variable and the environmental factors in each phase as explanatory variables. For instance, in general phase, f1, f2, f3, f4, f5, f6, f7 are explanatory variable, the reliability value is obtained from survey. Table 5 presents the significant environmental factors in each development phase. Backward elimination starts with all variables in each phase, eliminates the predictors according to the comparison between  $p$ -value and Alpha-to-Remove value, and stops when the  $p$ -value for all predictors is less than or equal to Alpha-to-Remove value (Montgomery et al., 2015). As seen from Table 5, most of the significant factors in general and analysis and design phase are also stay on Top 10 ranking EFs calculated by relative weighted method. It is very in-

**Table 5**  
Significant factors in each development phase.

| Development phase   | Significant factor | Description                       | Parameter estimate | p-value  |
|---------------------|--------------------|-----------------------------------|--------------------|----------|
| General             | f4                 | Amount of programming effort      | −0.074             | 0.0175   |
|                     | f5                 | Level of programming technologies | 0.09               | 0.0066   |
|                     | f7                 | Programming language              | 0.049              | 0.023    |
| Analysis and design | f11                | Requirement analysis              | 0.08               | 0.0026   |
| Coding              | f20                | Human nature                      | 0.092              | 6.11E−06 |
| Testing             | f23                | Testing resource allocation       | 0.099              | 0.0025   |

**Table 6**  
Comparison of ranking between multi-release software and single release software.

| Rank | Multi-release software (New ranking) |                                                | Single release software (Zhu et al., 2015) |                                                |
|------|--------------------------------------|------------------------------------------------|--------------------------------------------|------------------------------------------------|
|      | Factor                               | Description                                    | Factor                                     | Description                                    |
| 1    | f1                                   | Program complexity                             | f8                                         | Frequency of program specification change      |
| 2    | f11                                  | Requirement analysis                           | f22                                        | Testing effort                                 |
| 3    | f8                                   | Frequency of program specification change      | f21                                        | Testing environment                            |
| 4    | f22                                  | Testing effort                                 | f25                                        | Testing coverage                               |
| 5    | f12                                  | Relationship of detailed design to requirement | f1                                         | Program complexity                             |
| 6    | f4                                   | Amount of programming effort                   | f15                                        | Programmer skills                              |
| 7    | f25                                  | Testing coverage                               | f6                                         | percentage of reused modules                   |
| 8    | f18                                  | Program workload (stress)                      | f12                                        | Relationship of detailed design to requirement |
| 9    | f6                                   | Percentage of reused modules                   | f24                                        | Testing methodologies                          |
| 10   | f5                                   | Level of programming technologies              | f19                                        | Domain knowledge                               |

interesting to observe that human nature (f20) becomes a significant factor in coding phase. Human nature refers to the characteristics of software developer, such as the ability to avoid making mistakes. As the software release cycle becomes shorter, the software development cycle is also getting shorter accordingly. In such a scenario, the consequence of human nature on affecting software reliability is getting more attention in the development of multi-release software.

#### 4. Comparisons between single release software and Multi-release software

All of the previous studies (Zhang and Pham, 2000, 2001; Zhu et al., 2015) only focused on the impact of environmental factors on reliability for the development of single release software. Hence, one of the objectives for this paper is to compare the differences of environmental factors affecting software reliability between the development of single release software and multi-release software. These differences will be addressed from various aspects. For example, the ranking of environmental factors, the significance level of each environmental factor on software reliability assessment, principle components, the significance level of each development phase, the significant factors in each development phase.

##### 4.1. The ranking of environmental factors

There are 60% environmental factors on the top 10 ranking for single release software still stay on the top 10 ranking for multi-release software, however, the order of importance has changed. The detailed comparison of the top 10 ranking between single release software and multi-release software is presented in Table 6.

In order to stay competitive in the market, most of software company frequently release new updated versions of their software product. Depending on the purpose of the new release, some version may be released on a frequent basis, for instance, fix urgent faults in the previous software release; some version may be released on a long term basis since it will bring in major updates (Fritsch et al., 2010). Also, large software companies like Ericsson increasingly apply the principles of agile and lean software development in an iterative manner in order to quickly respond to

customers' feedback (Staron et al., 2012). It is understood that the importance level of testing phase for multi-release software is not as significant as it in single release software. Likewise, this study also supports this point of view. For example, as shown in Table 6, there are four environmental factors are coming from testing phase for single release software, while only two environmental factors are coming from testing phase for multi-release software.

Environmental factors, f11, f4, f18, f5, are showing in the top 10 ranking for the first time. f11, f4, f18, f5 refers to requirement analysis, amount of programming effort, program workload (stress), and level of programming technologies, respectively. Environmental factor, f11, requirement analysis, usually comes from customers, is fundamental for the following design and coding work. The specifications that software developers generate mostly based on the requirements from customers. It is also known that continuous delivery of customer requirement is critical for software development company in this very market-driven environment (Staron et al., 2012). Thus, it is reasonable that requirement analysis has significant impact on affecting software reliability in the development of multi-release software. f4 and f5 are both from general phase, are getting into top 10 ranking in this study. f4, amount of programming effort, which may directly improve the efficiency and reliability of the product. f5 refers to the level of programming technologies. The programming technologies are classified into four categories: design techniques, documentation techniques, programming techniques and development techniques. We can see that f5 illustrates a comprehensive framework of software development. Due to the short release of the software and the quick response to customer requirements, programming technologies have more significant impact on affecting the quality and reliability of multi-release software.

##### 4.2. Principle components for environmental factors

Principle component analysis is applied in both studies to determine the principle components to capture the characteristics of the top 10 ranking environmental factors. The top 10 ranking environmental factors are not the same for this two studies. Thus, some environmental factors, like f24, testing methodologies, f19, domain knowledge, f15, programmer skills, and f21, testing environment are not included in the data analysis of multi-release

**Table 7**

Comparison of principle components between multi-release software and single release software.

| Principle components for multi-release software |     |                                                | Principle components for single release software (Zhu et al., 2015) |     |                                                |
|-------------------------------------------------|-----|------------------------------------------------|---------------------------------------------------------------------|-----|------------------------------------------------|
| Component                                       | EFs | Description                                    | Component                                                           | EFs | Description                                    |
| PC1                                             | f12 | Relationship of detailed design to requirement | PC1                                                                 | f25 | Testing coverage                               |
|                                                 | f25 | Testing coverage                               |                                                                     | f21 | Testing environment                            |
|                                                 | f22 | Testing effort                                 |                                                                     | f22 | Testing effort                                 |
|                                                 | f11 | Requirement analysis                           |                                                                     | f24 | Testing methodologies                          |
|                                                 | f5  | Level of programming technologies              |                                                                     | f12 | Relationship of detailed design to requirement |
| PC2                                             | f18 | Program workload (Stress)                      | PC2                                                                 | f6  | Percentage of reused modules                   |
|                                                 | f4  | Amount of programming effort                   |                                                                     | f8  | Frequency of program specification change      |
| PC3                                             | f6  | Percentage of reused modules                   |                                                                     | f19 | Domain knowledge                               |
|                                                 | f1  | Program complexity                             | PC3                                                                 | f15 | Programmer skills                              |
| PC4                                             | f8  | Frequency of program specification change      |                                                                     | f1  | Program complexity                             |

**Table 8**

Comparison of final grouping.

| New grouping        |       |                | Grouping from Zhu et al. (2015) |       |                |
|---------------------|-------|----------------|---------------------------------|-------|----------------|
| Development phase   | Mean  | Final grouping | Development phase               | Mean  | Final grouping |
| General             | 4.540 | 1              | General                         | 5.225 | 1              |
| Analysis and design | 4.641 | 1              | Analysis and design             | 5.034 | 2              |
| Coding              | 4.470 | 1              | Coding                          | 4.933 | 2              |
| Testing             | 4.707 | 1              | Testing                         | 4.722 | 3              |

software. Four principle components are obtained in this paper for multi-release software, which can represent over 75% variation for the collected data; three principle component were generated in the previous findings for single release software, which can explain 69% variation of the collected data. The detailed comparisons are shown in Table 7. In summary, software developers need to choose the corresponding results according to their product requirements.

#### 4.3. Significance level for each development phase

Tukey method is utilized on both studies to compare the four development phases. The comparison of significance level of each development phase is shown in Table 8. The null hypothesis for both studies is all development phases have the same significance level in terms of their impact on software reliability assessment. During the development of single release software, three groups are categorized from previous study. Testing phase obviously gets the most important phase on affecting software reliability; analysis and design and coding phase have the same significance level; general phase has the least significance level. However, in the development of multi-release software, all of four development phases have the equal impact on affecting software reliability. As we discussed earlier, the application of agile and lean software development method results in short iteration and short release in order to quickly response customer feedback. Thus, the significance level of testing phase in the development of multi-release software is not as high as it in the development of single release software.

#### 4.4. Significant environmental factors in each development phase

Backward elimination method is applied in both studies to achieve the significant environment factors of each development phase for single/multi release software. The response for linear backward elimination method is the improvement on software reliability, which is described as percentage. The comparison for single/multi release software is illustrated in Table 9.

In both studies, we can see that most of the significant factors in each development phase are also on the top 10 ranking environmental factors. The significant factors in each development phase are different between single release software and multi-release software. For multi-release software, the significant factors

in general phase are f4, amount of programming effort, f5, level of programming technologies, and f7, programming language; while the significant factors in general phase for single release software are f4, amount of programming effort and f6, percentage of reused modules. Only f4, amount of programming effort, stays as significant factor in both studies. Analysis and design phase still have the similar choice of significant factors in this two studies since f11, requirement analysis, and f12, relationship of detailed design to requirement, both focus on the alignment of customer requirements.

In coding phase, the interesting finding is that multi-release software development focuses more on human nature due to the fact that quickly response to customer requirements leads to short release cycle, in which, how to avoid making mistake is crucial in the development process. In testing phase, testing resource allocation is significant in both studies. Technical-related environmental factors like testing methodology and coverage are more emphasized in the development of single release software. In summary, the selection of the significant factors in these two studies both reflects its product characteristics.

### 5. Other statistical learning method to select environmental factors

In this section, the main purpose is to select variables by the use of lasso regression and compare with the results from other variable selection methods discussed earlier. Why do we choose lasso regression to fit the model instead of least squares? It is the trade-off between the variance and bias due to the prediction accuracy and model interpretability (James et al., 2013). The least squares estimates tend to have low bias if the relationship between the response and the predictors is linear. If  $n \gg p$ , that is, if the number of observations,  $n$ , is much larger than the number of variables,  $p$ , the least squares tends to have low variance and performs well on the test observations. However, if  $n$  is not larger than  $p$ , then a lot of variability can be occurred in the least squares estimates. Over fitting and poor prediction can also appear in the future observations which are not used in the model training (James et al., 2013). In this study, we have collected 45 survey responses while 32 environmental factors are presented. The number of observations is not much larger than the number of variables. In or-

**Table 9**  
Comparison of significant factors.

| Significant factors for multi-release software |                    |                                   |          | Significant factors for single release software (Zhu et al., 2015) |                    |                                                |         |
|------------------------------------------------|--------------------|-----------------------------------|----------|--------------------------------------------------------------------|--------------------|------------------------------------------------|---------|
| Development phase                              | Significant factor | Description                       | p-value  | Development phase                                                  | Significant factor | Description                                    | p-value |
| General                                        | f4                 | Amount of programming effort      | 0.0175   | General                                                            | f4                 | Amount of programming effort                   | 0.0001  |
|                                                | f5                 | Level of programming technologies | 0.0066   |                                                                    | f6                 | Percentage of reused module                    | 0.013   |
|                                                | f7                 | Programming language              | 0.023    | Analysis and design                                                | f8                 | Frequency of program specification change      | 0.006   |
| Analysis and design                            | f11                | Requirement analysis              | 0.0026   |                                                                    | f12                | Relationship of detailed design to requirement | 0.014   |
|                                                | f20                | Human nature                      | 6.11E–06 |                                                                    | f15                | Programmer skills                              | 0.032   |
| Coding                                         | f23                | Testing resource allocation       | 0.0025   | Testing                                                            | f18                | Program workload (stress)                      | 0.0001  |
|                                                |                    |                                   |          |                                                                    | f23                | Testing resource allocation                    | 0.001   |
| Testing                                        |                    |                                   |          |                                                                    | f24                | Testing methodologies                          | 0.017   |
|                                                |                    |                                   |          |                                                                    | f25                | Testing coverage                               | 0.002   |

der to reduce the variance at the cost of increasing bias, we often shrink the estimated coefficients. Therefore, lasso regression, a shrinkage regression method, in which some coefficients can be estimated towards zero, hence, is applied to perform variable selection.

As the turning parameter  $\lambda$  is sufficiently large, some coefficients can be exactly equal to zero in lasso regression. For our survey dataset, 32 environmental factors are considered as predictors and reliability value is treated as response variable. By applying lasso regression, the non-zero-coefficient variables are f11, f20, f23, represent requirement analysis, human nature, and testing resource allocation, respectively. They are considered as significant predictors for affecting reliability in the development of multi-release software from lasso regression.

Different variable selection methods may provide different set of significant factors, but similarity could still exist in these sets. For instance, relative weighted method and backward elimination for each development phase both conclude f11, requirement analysis, is an important environmental factor. Moreover, f20, human nature, and f23, testing resource allocation, are selected as significant factor in each development phase addressed in Section 3. There are also some drawbacks on selecting variables using lasso regression. We can understand that other variables which are highly-correlated with the selected variables may be eliminated in the selection process based on the mathematical theory, however, some of them may possess more practical meaning than the selected variables. On this point of view, Table 4 can provide detailed explanation on their correlations.

## 6. Conclusions and remarks

Most literature only address the significant factors in the development of single release software. However, as the application of the principles of agile and lean software development, multi-release software becomes very common in the modern society. Hence, this study aims to investigate the impact of environmental factors on affecting software reliability assessment during the development of multi-release software. Researchers also can consider the significant environmental factors into software reliability modeling for multi-release software to better describe the impact of development environment. Comparisons between this study and previous findings regarding single and multi release software are also discussed.

1. 60% environmental factors listed on the top 10 ranking from previous study still stay on the top 10 ranking in this study. However, the order of importance has changed a lot. The importance level of testing phase in the development of multi-release software is not as significant as it in single release software because of the increasing application of agile software develop-

ment method. Correspondingly, factors related to testing phase are not important as before, while factors related to customer requirement/feedback analysis are getting more attention on affecting software reliability in the development of multi-release software.

2. In order to quickly response customers' feedback, short iteration and short release are required for the modern software. The significance level of testing phase for the development of multi-release software is not as high as it in the development of single release software. All of four development phases in multi-release software can be categorized as one group, while three groups are retained for single release software.
3. Significant factors in each development phase are not the same for single/multi release software. The weight of testing-related and programmer-skill-related factors decreases, as the weight of customer-related factors increases.
4. Other statistical learning methods, like lasso regression, is applied in the survey analysis. Requirement analysis, human nature, and testing resource allocation are considered as significant predictors on predicting reliability for the development of multi-release software. Different variable selection method has its own limitation, researchers and practitioners can choose the suitable results depends on their application.

In this study, survey responses from different industry/project/region are considered as one group to investigate the significance level of environmental factors on affecting software reliability in the development of multi-release software. Several research directions can be addressed in the future research. For example, the comparison of the conclusions amongst industry, project, or even region can be further discussed since the variance exists in those groups are considerably worthy to investigate.

## Acknowledgments

The authors would like to thank the editor and reviewers for their valuable comments and suggestions to improve this paper.

## Appendix A

### Table 4.

## Appendix B

### Definitions of the environmental factors

#### 1. Program complexity.

McCabe's V(G), Halstead's E and program size are well known complexity measures. There exists a significant relationship between each of these measures. However, it has been proved that

**Table 4**  
Correlation analysis for environmental factors.

| Factor | Description                                    | Correlated factor                                  | Pearson's r |
|--------|------------------------------------------------|----------------------------------------------------|-------------|
| f1     | Program complexity                             | f5 Level of programming technologies               | 0.452       |
| f11    | Requirement analysis                           | f22 Tesing effort                                  | 0.574       |
|        |                                                | f29 Storage devices                                | −0.565      |
|        |                                                | f5 Level of programming technologies               | 0.561       |
|        |                                                | f12 Relationship of detailed design to requirement | 0.553       |
|        |                                                | f28 Processors                                     | −0.539      |
|        |                                                | f23 Testing recourse allocation                    | 0.485       |
| f22    | Tesing effort                                  | f25 Testing coverage                               | 0.477       |
|        |                                                | f25 Testing coverage                               | 0.764       |
|        |                                                | f23 Testing recourse allocation                    | 0.684       |
|        |                                                | f26 Testing tools                                  | 0.679       |
|        |                                                | f21 Testing environment                            | 0.671       |
|        |                                                | f27 Documentation                                  | 0.632       |
|        |                                                | f5 Level of programming technologies               | 0.627       |
|        |                                                | f17 Development team size                          | 0.584       |
|        |                                                | f14 Development management                         | 0.573       |
| f12    | Relationship of detailed design to requirement | f10 Design methodology                             | 0.554       |
|        |                                                | f11 Requirement analysis                           | 0.553       |
|        |                                                | f6 Percentage of reused modules                    | 0.496       |
| f4     | Amount of programming effort                   | f3 Difficulty of programming                       | 0.646       |
|        |                                                | f15 Programmer skills                              | 0.524       |
|        |                                                | f7 Programming language                            | 0.476       |
| f25    | Testing coverage                               | f24 Testing methodologies                          | 0.801       |
|        |                                                | f22 Tesing effort                                  | 0.764       |
|        |                                                | f21 Testing environment                            | 0.712       |
|        |                                                | f26 Testing tools                                  | 0.704       |
|        |                                                | f23 Testing recourse allocation                    | 0.69        |
|        |                                                | f17 Development team size                          | 0.624       |
|        |                                                | f14 Development management                         | 0.57        |
|        |                                                | f13 Work standards                                 | 0.55        |
|        |                                                | f27 Documentation                                  | 0.539       |
|        |                                                | f5 Level of programming technologies               | 0.534       |
|        |                                                | f11 Requirement analysis                           | 0.477       |
|        |                                                | f2 Program categories                              | 0.463       |
|        |                                                | f32 System software                                | 0.451       |
| f18    | Program workload (stress)                      | f20 Human nature                                   | 0.497       |
|        |                                                | f19 Domain knowledge                               | 0.491       |
|        |                                                | f17 Development team size                          | 0.467       |
| f6     | Percentage of reused modules                   | f12 Relationship of detailed design to requirement | 0.496       |
|        |                                                | f5 Level of programming technologies               | 0.484       |
| f5     | Level of programming technologies              | f22 Tesing effort                                  | 0.627       |
|        |                                                | f11 Requirement analysis                           | 0.561       |
|        |                                                | f25 Testing coverage                               | 0.534       |
|        |                                                | f26 Testing tools                                  | 0.510       |
|        |                                                | f6 Percentage of reused modules                    | 0.484       |
|        |                                                | f23 Testing recourse allocation                    | 0.474       |
|        |                                                | f1 Program complexity                              | 0.452       |
| f15    | Programmer skills                              | f7 Programming language                            | 0.647       |
|        |                                                | f19 Domain knowledge                               | 0.603       |
|        |                                                | f16 Programmer organization                        | 0.591       |
|        |                                                | f4 Amount of programming effort                    | 0.524       |
| f21    | Testing environment                            | f25 Testing coverage                               | 0.711       |
|        |                                                | f22 Tesing effort                                  | 0.671       |
|        |                                                | f23 Testing recourse allocation                    | 0.643       |
|        |                                                | f17 Development team size                          | 0.627       |
|        |                                                | f26 Testing tools                                  | 0.57        |
|        |                                                | f20 Human nature                                   | 0.538       |
|        |                                                | f24 Testing methodologies                          | 0.475       |
| f23    | Testing recourse allocation                    | f26 Testing tools                                  | 0.842       |
|        |                                                | f25 Testing coverage                               | 0.69        |
|        |                                                | f22 Tesing effort                                  | 0.685       |
|        |                                                | f24 Testing methodologies                          | 0.668       |
|        |                                                | f21 Testing environment                            | 0.643       |
|        |                                                | f17 Development team size                          | 0.558       |
|        |                                                | f14 Development management                         | 0.555       |
|        |                                                | f9 Volume of program design documents              | 0.492       |
|        |                                                | f11 Requirement analysis                           | 0.485       |
|        |                                                | f13 Work standards                                 | 0.479       |
|        |                                                | f5 Level of programming technologies               | 0.474       |
| f10    | Design methodology                             | f14 Development management                         | 0.604       |
|        |                                                | f9 Volume of program design documents              | 0.563       |
|        |                                                | f12 Relationship of detailed design to requirement | 0.554       |

(continued on next page)

Table 4 (continued)

| Factor | Description               | Correlated factor                     | Pearson's r |
|--------|---------------------------|---------------------------------------|-------------|
| f24    | Testing methodologies     | f25 Testing coverage                  | 0.8         |
|        |                           | f13 Work standards                    | 0.715       |
|        |                           | f14 Development management            | 0.694       |
|        |                           | f23 Testing resource allocation       | 0.668       |
|        |                           | f22 Testing effort                    | 0.625       |
|        |                           | f26 Testing tools                     | 0.621       |
|        |                           | f9 Volume of program design documents | 0.613       |
|        |                           | f17 Development team size             | 0.599       |
|        |                           | f27 Documentation                     | 0.564       |
|        |                           | f32 System software                   | 0.508       |
| f19    | Domain knowledge          | f31 Telecommunication devices         | 0.47        |
|        |                           | f29 Storage devices                   | 0.741       |
|        |                           | f7 Programming language               | 0.653       |
|        |                           | f16 Programmer organization           | 0.631       |
|        |                           | f15 Programmer skills                 | 0.603       |
|        |                           | f20 Human nature                      | 0.564       |
|        |                           | f17 Development team size             | 0.513       |
|        |                           | f18 Program workload (stress)         | 0.491       |
| f27    | Documentation             | f14 Development management            | 0.642       |
|        |                           | f22 Testing effort                    | 0.632       |
|        |                           | f24 Testing methodologies             | 0.564       |
|        |                           | f13 Work standards                    | 0.551       |
|        |                           | f25 Testing coverage                  | 0.539       |
|        |                           | f31 Telecommunication devices         | 0.512       |
|        |                           | f17 Development team size             | 0.493       |
| f3     | Difficulty of programming | f4 Amount of programming effort       | 0.646       |
|        |                           | f7 Programming language               | 0.538       |
| f26    | Testing tools             | f23 Testing resource allocation       | 0.843       |
|        |                           | f25 Testing coverage                  | 0.703       |
|        |                           | f22 Testing effort                    | 0.679       |
|        |                           | f24 Testing methodologies             | 0.621       |
|        |                           | f21 Testing environment               | 0.57        |
|        |                           | f5 Level of programming technologies  | 0.51        |
|        |                           | f32 System software                   | 0.492       |
|        |                           | f17 Development team size             | 0.459       |
| f14    | Development management    | f2 Program categories                 | 0.451       |
|        |                           | f24 Testing methodologies             | 0.694       |
|        |                           | f27 Documentation                     | 0.642       |
|        |                           | f10 Design methodology                | 0.604       |
|        |                           | f22 Testing effort                    | 0.573       |
|        |                           | f23 Testing resource allocation       | 0.554       |
|        |                           | f13 Work standards                    | 0.498       |
| f16    | Programmer organization   | f19 Domain knowledge                  | 0.631       |
|        |                           | f31 Telecommunication devices         | 0.611       |
|        |                           | f29 Storage devices                   | 0.596       |
|        |                           | f15 Programmer skills                 | 0.592       |
|        |                           | f17 Development team size             | 0.522       |
|        |                           | f28 Processors                        | 0.504       |
|        |                           | f13 Work standards                    | 0.715       |
|        |                           | f9 Volume of program design documents | 0.472       |
| f2     | Program categories        | f25 Testing coverage                  | 0.463       |
|        |                           | f26 Testing tools                     | 0.451       |
|        |                           | f19 Domain knowledge                  | 0.564       |
| f20    | Human nature              | f7 Programming language               | 0.563       |
|        |                           | f21 Testing environment               | 0.538       |
|        |                           | f18 Program workload (stress)         | 0.497       |
|        |                           | f32 System software                   | 0.663       |
| f17    | Development team size     | f21 Testing environment               | 0.627       |
|        |                           | f24 Testing methodologies             | 0.599       |
|        |                           | f22 Testing effort                    | 0.584       |
|        |                           | f31 Telecommunication devices         | 0.572       |
|        |                           | f23 Testing resource allocation       | 0.560       |
|        |                           | f30 Input/output devices              | 0.56        |
|        |                           | f15 Programmer skills                 | 0.55        |
|        |                           | f16 Programmer organization           | 0.522       |
|        |                           | f19 Domain knowledge                  | 0.513       |
|        |                           | f27 Documentation                     | 0.493       |
|        |                           | f14 Development management            | 0.483       |
|        |                           | f7 Programming language               | 0.477       |
|        |                           | f18 Program workload (stress)         | 0.467       |
|        |                           | f26 Testing tools                     | 0.459       |

(continued on next page)

Table 4 (continued)

| Factor | Description                        | Correlated factor                     | Pearson's r |
|--------|------------------------------------|---------------------------------------|-------------|
| f9     | Volume of program design documents | f2 Program categories                 | 0.472       |
|        |                                    | f10 Design methodology                | 0.563       |
|        |                                    | f13 Work standards                    | 0.584       |
|        |                                    | f14 Development management            | 0.500       |
|        |                                    | f23 Testing resource allocation       | 0.492       |
| f13    | Work standards                     | f24 Testing methodologies             | 0.613       |
|        |                                    | f2 Program categories                 | 0.715       |
|        |                                    | f24 Testing methodologies             | 0.715       |
|        |                                    | f9 Volume of program design documents | 0.584       |
|        |                                    | f27 Documentation                     | 0.551       |
| f28    | Processors                         | f25 Testing coverage                  | 0.550       |
|        |                                    | f14 Development management            | 0.498       |
|        |                                    | f23 Testing resource allocation       | 0.479       |
|        |                                    | f29 Storage devices                   | 0.887       |
|        |                                    | f30 Input/output devices              | 0.668       |
| f32    | System software                    | f31 Telecommunication devices         | 0.561       |
|        |                                    | f16 Programmer organization           | 0.506       |
|        |                                    | f19 Domain knowledge                  | 0.469       |
|        |                                    | f7 Programming language               | 0.459       |
|        |                                    | f11 Requirement analysis              | −0.539      |
| f29    | Storage devices                    | f17 Development team size             | 0.663       |
|        |                                    | f30 Input/output devices              | 0.654       |
|        |                                    | f31 Telecommunication devices         | 0.65        |
|        |                                    | f25 Testing coverage                  | 0.571       |
|        |                                    | f24 Testing methodologies             | 0.508       |
| f7     | Programming language               | f26 Testing tools                     | 0.492       |
|        |                                    | f14 Development management            | 0.474       |
|        |                                    | f22 Testing effort                    | 0.467       |
|        |                                    | f28 Processors                        | 0.886       |
|        |                                    | f30 Input/output devices              | 0.683       |
| f30    | Input/output devices               | f31 Telecommunication devices         | 0.619       |
|        |                                    | f16 Programmer organization           | 0.596       |
|        |                                    | f7 Programming language               | 0.503       |
|        |                                    | f19 Domain knowledge                  | 0.471       |
|        |                                    | f11 Requirement analysis              | −0.565      |
| f31    | Telecommunication devices          | f4 Amount of programming effort       | 0.476       |
|        |                                    | f15 Programmer skills                 | 0.647       |
|        |                                    | f17 Development team size             | 0.477       |
|        |                                    | f19 Domain knowledge                  | 0.653       |
|        |                                    | f20 Human nature                      | 0.563       |
| f14    | Development management             | f28 Processors                        | 0.459       |
|        |                                    | f29 Storage devices                   | 0.503       |
|        |                                    | f30 Input/output devices              | 0.467       |
|        |                                    | f31 Telecommunication devices         | 0.829       |
|        |                                    | f29 Storage devices                   | 0.683       |
| f29    | Storage devices                    | f28 Processors                        | 0.668       |
|        |                                    | f32 System software                   | 0.654       |
|        |                                    | f17 Development team size             | 0.56        |
|        |                                    | f16 Programmer organization           | 0.497       |
|        |                                    | f7 Programming language               | 0.468       |
| f16    | Programmer organization            | f30 Input/output devices              | 0.839       |
|        |                                    | f32 System software                   | 0.65        |
|        |                                    | f29 Storage devices                   | 0.619       |
|        |                                    | f16 Programmer organization           | 0.612       |
|        |                                    | f17 Development team size             | 0.572       |
| f28    | Processors                         | f28 Processors                        | 0.561       |
|        |                                    | f14 Development management            | 0.488       |

none of these seemed significantly better than program size. For this reason, program size (Kiloline of code: KLOC) is used as a measure of program complexity. The level of this factor is “High” means the program size is greater than 50 KLOC, otherwise the level is “Low”.

#### 2. Program categories.

The program categories indicate system complexity. There are four program categories: operating system, communication control program, data base management system and languages processor.

#### 3. Difficulty of programming (PDIF).

Difficulty of programming is defined according to Putnum as follows:

$$PDIF = \frac{k}{t^2} (\text{man years}/\text{years}^2)$$

where  $k$  is the amount of programming effort,  $t$  is the amount of programming time.

#### 4. Amount of Programming effort.

The deliberate programming effort may be regarded as effective for reducing the number of errors made. This is calculated in man years.

#### 5. Level of programming technologies (TLVL).

The programming technologies are classified into four categories: design techniques, documentation techniques, programming techniques (including programming languages) and development computer access environment. Each of the categories had a rating scale (low, middle, and high) and the rating scores  $T=0.8, 1, 1.2$ , respectively, are allocated. These rating scores were determined by referring to Boehm's cost driver productivity range. The

final TLVL is computed as follows by using the rating scores:

$$TLVL = \sum_{i=1}^4 T_i$$

where  $T_i$  is the rating score of category  $i$ .

#### 6. Percentage of reused code/modules (PORC).

When people develop some new software products or when they update the old version of their software products, they usually keep some of the module of the code which can be reused, and add in some new ones. That is why the measure is important to include.

$$PORC = \frac{S_0}{S_N + S_0}$$

where  $S_0$  represents kiloline of code for the existing module and  $S_N$  represents kiloline of code for new modules.

#### 7. Programming language.

Different programming languages have different complexity and structure, therefore the possibility for different languages to introduce errors are different.

#### 8. Frequency of program specification change (SCHG).

SCHG is calculated from the number of pages of problem reports generated to change the program design specifications during programming phase of development.

#### 9. Volume of program design documents (DOCC).

Program design documents that lack sufficient contents for the program produce errors. DOCC is calculated from the number of pages of new and modified program design documents.

#### 10. Design methodologies.

Different design methodologies for the same software may have different impact on the quality of the final software products. There are two types of design methodologies: structured design and functional design.

#### 11. Requirements analysis.

Requirements usually come from customers. Based on the requirements software developers generate specifications. Usually customers and developers meet to verify the requirements and achieve understanding of the problems. Requirement analysis is fundamental for the following design and coding work.

#### 12. Relationship of detailed design to requirement.

At the end of design phase, detailed design is compared with requirements. Inspections are performed to verify whether the functions designed meet the requirements. Modifications can be made to remove the misunderstanding between the customers and developers.

#### 13. Work standards.

Work standard is the norm the developing team needs to obey. This could be company standard or group standard. Work standard indicates products to be made at each phase, design document format, design document description level and content, and the items to be checked in verifying the design documents. Having this kind of norms or not has impact on the quality of software products.

#### 14. Development management.

Development management includes all the organization and decision-making activities. From the specification phase to design, code, and testing and even operational phase, development managers schedule the meeting time, keep all participants in touch and keep track of the development progress and work standards, give instructions, make decisions.

#### 15. Programmer skills (PSKL).

PSKL has direct impact on the reliability of software products. PSKL can be defined as the average number of years of programming experience of programmers.

$$PSKL = \frac{\sum_{i=1}^n I_i}{n} (\text{years})$$

where  $I_i$  is the number of years' experience of programmer  $i$ , and  $n$  is the total number of programmers.

#### 16. Programmer organization (ICON).

ICON is defined as the percentage of high-quality programmers. ICON is computed as follows:

$$ICON = \frac{n_h}{n}$$

where  $n_h$  is the number of programmer whose programming experience is more than 6 years, and  $n$  is the total number of programmers.

#### 17. Development team size.

Team size has impact on the quality of software products. There are some people believe that large team size will improve the quality of the software since there are more people involving on the development process. But others claim that smaller but experienced development team will be better.

#### 18. Program work contents (Stress).

During the software development, stress factors in terms of "work contents" such as schedule pressure and too much work are the major factors. This includes developer's mental stress and physical stress. Stress can be classified into several degree groups.

#### 19. Domain knowledge.

Domain knowledge refers to the programmer's knowledge of the input space and output target. Insufficient knowledge may cause problems in coding and testing procedures.

#### 20. Human nature.

This refer to the developers' characteristics, including the ability to avoid making the working mistakes, careless work omission.

#### 21. Testing environment.

In order to find out more errors during testing phase, testing environment should mimic the operational environment. This can be defined as the degree of the compatibility of testing and operational environments.

#### 22. Testing effort.

Testing effort can be defined as the number of testing cases generated, testing expenditures, or human years testing takes.

#### 23. Testing resource allocation.

Testing resource allocation refers to different schemes to allocate the testing resources, in terms of testers, facilities and schedules of the testing activities.

#### 24. Testing methodologies.

Different testing methodologies have different impact on the quality of software products. Good testing methodologies may test more paths and need less time.

#### 25. Testing coverage.

Test coverage is defined as the percentage of the source code which are covered by the test cases. It can be expressed as:

$$TCVG = \frac{S_c}{S_T}$$

where  $S_c$  represents kiloline of code which is covered by testing, and  $S_T$  represents total kiloline of code

#### 26. Testing tools.

There exist many different testing tools. These include the software packages testers utilize to carry out the testing tasks. Different tools also provide different quality and testing measures.

#### 27. Documentation.

Documentation includes all paperwork from specification to design, coding and testing. This can serve as resources for developers to allocate changes and problems; for programmers to review the codes and for testers to examine the codes and detect bugs.

Factors 28–32 are the hardware systems used for software development. The definite is obvious.

## References

- Adolph, S., Kruchten, P., Hall, W., 2012. Reconciling perspectives: a grounded theory of how people manage the process of software development. *J. Syst. Software* 85 (6), 1269–1286.
- Al-Emran, A., Pfahl, D., 2007. Operational planning, re-planning and risk analysis for software releases. In: *International Conference on Product Focused Software Process Improvement*. Springer Berlin Heidelberg, pp. 315–329.
- Beck, K., Beedle, M., Van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., 2001. Manifesto for agile software development. [www.agilemanifesto.org](http://www.agilemanifesto.org).
- Chow, T., Cao, D.B., 2008. A survey study of critical success factors in agile software projects. *J. Syst. Software* 81 (6), 961–971.
- Clarke, P., O'Connor, R.V., 2012. The situational factors that affect the software development process: towards a comprehensive reference framework. *Inf. Software Technol.* 54 (5), 433–447.
- Etgar, R., Gelbard, R., Cohen, Y., 2017. Optimizing version release dates of research and development long-term processes. *Eur. J. Oper. Res.* 259, 642–653.
- Febrero, F., Calero, C., Moraga, M.A., 2016. Software reliability modeling based on ISO/IEC SQuaRE. *Inf. Software Technol.* 70, 18–29.
- Fritsch, B.L., Bibr, V., Blagojevic, V., Goring, B.R., Shenfield, M., Vitanov, K.B., 2010. U.S. Patent No. 7,747,995. Washington, DC: U.S. Patent and Trademark Office.
- Gorschek, T., Davis, A.M., 2008. Requirements engineering: In search of the dependent variables. *Inf. Software Technol.* 50 (1), 67–75.
- Hu, Q.P., Peng, R., Xie, M., Ng, S.H., Levitin, G., 2011. Software reliability modelling and optimization for multi-release software development processes. In: *Proceedings of International Conference on Industrial Engineering and Engineering Management (IEEM)*. IEEE, pp. 1534–1538.
- James, G., Witten, D., Hastie, T., Tibshirani, R., 2013. *An Introduction to Statistical Learning* (vol. 6). New York: Springer.
- Kapur, P.K., Pham, H., Anand, S., Yadav, K., 2011. A unified approach for developing software reliability growth models in the presence of imperfect debugging and error generation. *IEEE Trans. Reliab.* 60 (1), 331–340.
- Li, L., Harman, M., Letier, E., Zhang, Y., 2014. Robust next release problem: handling uncertainty during optimization. In: *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*. ACM, pp. 1247–1254.
- Maurice, S., Ruhe, G., Saliu, O., 2006. Decision support for value-based software release planning. In: *Value-Based Software Engineering*. Springer, Berlin Heidelberg, pp. 247–261.
- Misra, S.C., Kumar, V., Kumar, U., 2009. Identifying some important success factors in adopting agile software development practices. *J. Syst. Software* 82 (11), 1869–1890.
- Mohanty, R., Ravi, V., Patra, M.R., 2013. Hybrid intelligent systems for predicting software reliability. *Appl. Soft Comput.* 13 (1), 189–200.
- Montgomery, D.C., Peck, E.A., Vining, G.G., 2015. *Introduction to Linear Regression Analysis*. John Wiley & Sons.
- Ruhe, G., Momoh, J., 2005, January. Strategic release planning and evaluation of operational feasibility. In: *Proceedings of the 38th Annual Hawaii International Conference on System Sciences*. IEEE, p. 313.
- Sahoo, S.K., Criswell, J., Adve, V., 2010, May. An empirical study of reported bugs in server software with implications for automated bug diagnosis. In: *2010 ACM/IEEE 32nd International Conference on Software Engineering*, 1. IEEE, pp. 485–494.
- Saliu, O., Ruhe, G., 2005. Software release planning for evolving systems. *Innovations Syst. Software Eng.* 1 (2), 189–204.
- Staron, M., Meding, W., Palm, K., 2012, May. Release readiness indicator for mature agile and lean software development projects. In: *International Conference on Agile Software Development*. Springer, Berlin Heidelberg, pp. 93–107.
- Svahnberg, M., Gorschek, T., Feldt, R., Torkar, R., Saleem, S.B., Shafique, M.U., 2010. A systematic review on strategic release planning models. *Inf. Software Technol.* 52 (3), 237–248.
- Szöke, Á., 2011. Conceptual scheduling model and optimized release scheduling for agile environments. *Inf. Software Technol.* 53 (6), 574–591.
- Yang, J., Liu, Y., Xie, M., Zhao, M., 2016. Modeling and analysis of reliability of multi-release open source software incorporating both fault detection and correction processes. *J. Syst. Software* 115, 102–110.
- Zhu, M., Zhang, X., Pham, H., 2015. A comparison analysis of environmental factors affecting software reliability. *J. Syst. Software* 109, 150–160.
- Zhang, X., Pham, H., 2000. An analysis of factors affecting software reliability. *J. Syst. Software* 50 (1), 43–56.
- Zhang, X., Shin, M.Y., Pham, H., 2001. Exploratory analysis of environmental factors for enhancing the software reliability assessment. *J. Syst. Software* 57 (1), 73–78.

**Mengmeng Zhu** is a Ph.D. candidate of the Department of Industrial and Systems Engineering at Rutgers University. She received the B.S. degree in Industrial Engineering from Nanjing University of Technology and the M.S. degree in Industrial Engineering from Rutgers University. She has published 4 journal papers including in *Annals of Operations Research* and *Journal of Systems and Software*. Her research interests include software reliability modeling and failure data analysis.

**Hoang Pham** is a Distinguished Professor and former Chairman of the Department of Industrial and Systems Engineering at Rutgers University. He is the author or coauthor of five books and has published over 140 journal articles, and edited 10 books including *Springer Handbook in Engineering Statistics* and *Handbook in Reliability Engineering*. He has been served as editor-in-chief, editor, associate editor, guest editor and board member of many journals. He is the editor of *Springer Book Series in Reliability Engineering* and editor of *World Scientific Book Series on Industrial and Systems Engineering*. His numerous awards include the 2009 IEEE Reliability Society *Engineer of the Year Award*. He is a Fellow of the IEEE and IIE.